



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Летучка № 6
по курсу «Языки и методы программирования»

«Дженерики в Java. Stack<T>»

Студент группы ИУ9-22Б
Булдаков А. С.

Преподаватель
Посевин Д. П.

Москва 2026

Содержание

Цель работы	1
Теоретические сведения	1
Реализация	1
Универсальный класс Stack	1
Класс RVector	2
Класс Scalar	3
Класс Vector	3
Классы Body и Universe	4
Использование дженериков	6
Тестирование	7
Тест 1: Скалярные произведения	7
Тест 2: Стеки с разными типами	8
Вывод	8

Цель работы

Изучить механизм дженериков (generic types) в Java. Реализовать универсальный класс Stack для работы с различными типами данных. Применить дженерики для создания стека, содержащего объекты различного типа.

Задачи:

- Изучить синтаксис дженериков в Java.
- Реализовать универсальный класс Stack с методами push() и pop().
- Создать классы RVector и Scalar для работы с векторами.
- Применить дженерики для хранения различных типов в стеке.

Теоретические сведения

Дженерики (generics) — это механизм, позволяющий создавать классы и методы, которые работают с различными типами данных, сохраняя при этом безопасность типов во время компиляции.

Основные преимущества дженериков:

- Безопасность типов — ошибки обнаруживаются на этапе компиляции
- Уменьшение дублирования кода — один класс работает с разными типами
- Упрощение кода — не требуется приведение типов

Синтаксис дженериков:

- `class Stack<T>` — параметр типа T
- `Stack<Universe>` — конкретный тип

Реализация

Универсальный класс Stack

Класс Stack реализует стек с динамическим расширением:

Main.java

```
class Stack<T> {
    private int coun = 0;
    private Object[] buf = new Object[16];

    public boolean empty() {
        return coun == 0;
    }

    public void push(T x) {
        if (coun == buf.length) {
            buf = Arrays.copyOf(buf, buf.length * 2);
        }
        buf[coun++] = x;
    }

    @SuppressWarnings("unchecked")
    public T pop() {
        if (empty()) {
            throw new RuntimeException("underflow");
        }
        return (T) buf[--coun];
    }
}
```

Методы:

- empty() — проверка пустоты стека
- push(T x) — добавление элемента с автоматическим расширением
- pop() — извлечение элемента

Класс RVector

Класс для представления радиус-вектора:

Main.java

```
class RVector {
    int x, y, z;

    public RVector(int x, int y, int z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }
}
```

Класс Scalar

Класс для вычисления скалярного произведения векторов:

Main.java

```
class Scalar {
    RVector a, b;

    public Scalar(RVector a, RVector b) {
        this.a = a;
        this.b = b;
    }

    public int Evaluate() {
        return a.x * b.x + a.y * b.y + a.z * b.z;
    }
}
```

Скалярное произведение:

$$ac \cdot b = a_x c \cdot b_x + a_y c \cdot b_y + a_z c \cdot b_z$$

Класс Vector

Класс Vector для трёхмерного пространства (из предыдущих работ):

Vector.java

```
public class Vector {
    public double x, y, z;

    public Vector(double x, double y, double z) {
        this.x = x;
        this.y = y;
        this.z = z;
    }

    public Vector add(Vector v) {
        return new Vector(x + v.x, y + v.y, z + v.z);
    }

    public static double dist(Vector a, Vector b) {
        double dx = a.x - b.x;
        double dy = a.y - b.y;
        double dz = a.z - b.z;
        return sqrt(dx * dx + dy * dy + dz * dz);
    }

    public static double dot(Vector a, Vector b) {
        return a.x * b.x + a.y * b.y + a.z * b.z;
    }

    public double length() {
        return dist(new Vector(0, 0, 0), this);
    }
}
```

Классы Body и Universe

Классы для моделирования гравитационного взаимодействия:

Body.java

```
public class Body {
    Vector r;
    Vector F;
    double m;
    String name;

    public static int instance_count;
    public static final double G = 6.674e-11;

    public Vector getForceFrom(Body other) {
        double r = Vector.dist(this.r, other.r);
        double amount = G * this.m * other.m / (r * r);
        double dx = other.r.x - this.r.x;
        double dy = other.r.y - this.r.y;
        double dz = other.r.z - this.r.z;
        return new Vector(amount * dx / r, amount * dy / r, amount *
dz / r);
    }
}
```

Universe.java

```
public class Universe {
    Body[] bodies;

    public Universe(int n) {
        bodies = new Body[n];
        for (int i = 0; i < n; i++) {
            bodies[i] = new Body();
        }
    }

    public double getTotalMass() {
        double total = 0;
        for (Body bud : bodies) {
            total += bud.m;
        }
        return total;
    }

    public Vector getResultForce(int idx) {
        Vector v = new Vector(0.0, 0.0, 0.0);
        for (int i = 0; i < bodies.length; i++) {
            if (i == idx)
                continue;
            v = v.add(bodies[idx].getForceFrom(bodies[i]));
        }
        return v;
    }
}
```

Использование дженериков

==Основная программа

Main.java

```
public class Main {
    public static double drain(Stack<Universe> s) {
        double totalMass = 0.0;
        while (!s.empty()) {
            for (Body b : s.pop().bodies) {
                totalMass += b.m;
            }
        }
        return totalMass;
    }

    public static void main(String[] args) {
        RVector v1 = new RVector(1, 2, 3);
        RVector v2 = new RVector(4, 5, 6);
        RVector v3 = new RVector(7, 8, 9);
        RVector v4 = new RVector(10, 11, 12);

        Stack<Scalar> stack = new Stack<>();
        stack.push(new Scalar(v1, v2));
        stack.push(new Scalar(v3, v4));

        Scalar s1 = stack.pop();
        Scalar s2 = stack.pop();

        int result = s1.Evaluate() + s2.Evaluate();
        System.out.println(result);

        Stack<Universe> stack2 = new Stack<>();
        stack2.push(new Universe(5));
        stack2.push(new Universe(5));
        System.out.println(drain(stack2));
    }
}
```

Тестирование

Тест 1: Скалярные произведения

Вычисление скалярных произведений:

- $v_1c \cdot v_2 = 1c \cdot 4 + 2c \cdot 5 + 3c \cdot 6 = 4 + 10 + 18 = 32$
- $v_3c \cdot v_4 = 7c \cdot 10 + 8c \cdot 11 + 9c \cdot 12 = 70 + 88 + 108 = 266$
- Сумма: $32 + 266 = 298$

terminal

```
$ java Main
298
[+] Body class loaded
[*] Universe class loaded
[+] new Universe instance created ...
```

Тест 2: Стеки с разными типами

Один и тот же класс Stack используется с разными типами:

- Stack<Scalar> — стек скалярных произведений
- Stack<Universe> — стек вселенных

Main.java

```
public static double drain(Stack<Universe> s) {
    double totalMass = 0.0;
    while (!s.empty()) {
        for (Body b : s.pop().bodies) {
            totalMass += b.m;
        }
    }
    return totalMass;
}
```

Результат — суммарная масса всех тел во вселенных:

Вывод

В ходе лабораторной работы были изучены и применены дженерики в Java:

1. Реализован универсальный класс Stack<T> с методами push() и pop()
2. Класс работает с любым типом данных, сохраняя безопасность типов
3. Продемонстрировано использование одного класса с разными типами:
 - Stack<Scalar> — для скалярных произведений
 - Stack<Universe> — для объектов Вселенной
4. Динамическое расширение стека при заполнении

Дженерики позволяют создавать универсальный код, который работает с различными типами данных без дублирования.